

Ethan Ho

September 16, 2025

CMPE 252, Sec 01

Professor Liu

Homework 1

## Dijkstras Implementation

### Algorithm Implementation:

```
### Dijkstra's Algorithm ###
dist = [math.inf] * (n+1)
dist[start] = 0
visited = [False] * (n+1)

for _ in range(n):
    best = math.inf
    u = None
    for i in range(1, n+1):
        if not visited[i] and dist[i] < best:
            best = dist[i]
            u = i

    if u is None:
        break
    visited[u] = True

    for v, w in adjacencyList[u]:
        if not visited[v] and (dist[u] + w < dist[v]):
            dist[v] = dist[u] + w
```

Above is my implementation of Dijkstra's algorithm before adding any visualization. The algorithm essentially relies on filling the dist array, which represents the distances to all nodes, with the minimum cost paths. I also kept a visited set in order to check in constant time, as when I traverse from A -> B, I do not want to check B -> A again. To cover all nodes, I iterate n times, where n is the number of vertices given from the input file, and check all the costs traveling from

the current node,  $u$ , and update  $dist$ . After that, I then do the same to the neighboring nodes before moving on to the next  $u$ .

### Visualization:

For my visualization, I used PyGame in order to animate the traversal, as matplotlib took a long time to capture an image at each step and compile it into a video. I did accidentally misread the instructions and implemented it so that a video would play when you run the program; however, I later found out that it should instead generate an MP4. I left that option to pop up as an optional argument you could enter, but by default, it is off. In addition, my implementation of Dijkstra's conceptually is the same, but in order to animate the steps taken, I had to take captures of the state of the grid at each step, as seen below. This state not only helps me to generate the MP4 at the end but also to retrace my steps and see what the optimal path was from my start node to my end node.

```
yield {  
    "current": u,  
    "neighbors": nbrs,  
    "traversed_edges": set(traversed),  
    "dist": dist[:],  
    "prev": prev[:],  
    "visited": visited[:],  
}
```

**Dependencies and Compilation:**

My project requires the following libraries, which could be installed with pip: pygame and imageio. In order to compile the program, simply run:

```
python3 017157582.py
```

After running, it should generate an MP4 as well as a txt folder with the given ID. In addition, I have the following optional arguments that may be included for convenience:

|          |                                                                |
|----------|----------------------------------------------------------------|
| --coords | Path to the coords.txt. Default set to the current directory.  |
| --input  | Path to the input.txt. Default set to the current directory.   |
| --width  | Window width in pixels.                                        |
| --height | Window height in pixels.                                       |
| --popup  | Pops open a window to visualize the algorithm. Default is off. |

It is recommended to stick with simply running the program as is unless you run into any visual issues.

**Example Video:**

Below is the sample video that should be generated when running the program.

@ 📺 017157582.mp4